



Sliding Window Problem Solving

- Gaurish Baliga

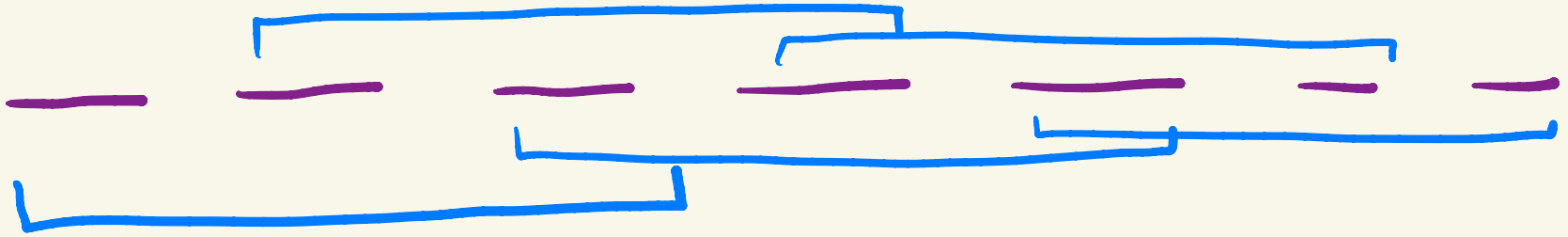
Problem 1



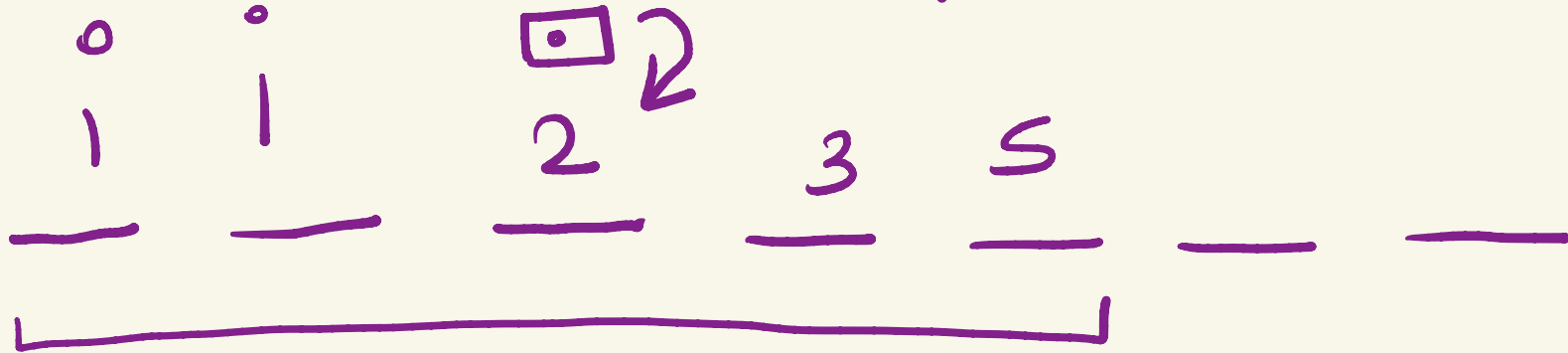
Sliding Window Median:

<https://leetcode.com/problems/sliding-window-median>

Find median for each window



→ multiset



k elements

$$\left\lfloor \frac{k}{2} \right\rfloor + 1$$

→ multiset

→ each window → $\frac{k}{2} + 1$ times

$$O(n \cdot \log k + nk)$$

$$n \log k + n k$$



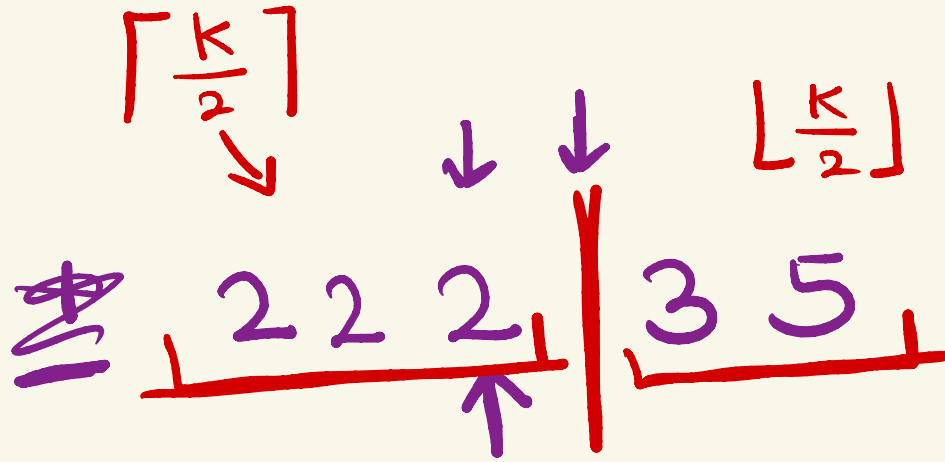
TLE



$$O(n(k + \log k)) \rightarrow O(n^2)$$

$$k \rightarrow n$$

* middle element of a multiset



2 multisets

median



$\lceil \frac{K}{2} \rceil$

$\lfloor \frac{K}{2} \rfloor$

$$\frac{x+y}{2}$$

X

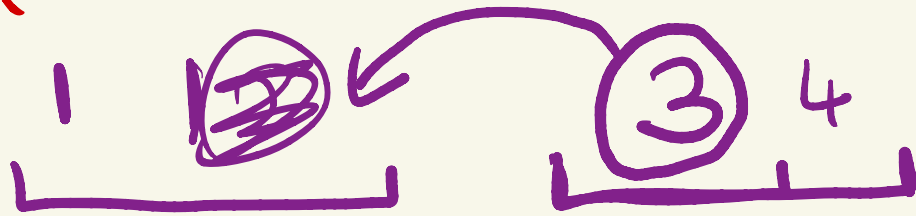
$\lceil \frac{K}{2} \rceil$ $\lfloor \frac{K}{2} \rfloor$
left



1 3 4



1 3 4 5



- left +
+ left

- left +
+ right

- right
+ left

- right
+ right

Removals

X
[]

X
[]

- right $\rightarrow$ $x \quad x-1$

- left $\rightarrow$ $x-1 < \textcircled{x}$
 $\rightarrow$ invalid

$\rightarrow$
 $x \quad x-1$

$\hookrightarrow$ size of the multiset

$x+1 \quad x$
 $\rightarrow$ []

- right $\rightarrow$ $x+1 \quad x-1$
 $\rightarrow$ invalid

$x \quad x$

- Left $x \quad x$

Insert

x x

└────────────────────────────────┘

+ left

$x+1$ x ✓

└──────────┘

+ right

x $x+1$

└──────────┘

invalid

└──────────┘

$x+1$ x

$x+1$ x

└────────────────────────────────┘

+ left

$x+2$ x

└──────────┘

invalid

x

└──────────┘

$x+1$ $x+1$

+ right

$x+1$ $x+1$

Invalid conditions:

→ $\text{right.size()} > \text{left.size()}$

⇒ shift smallest from right
to left

→ $\text{left.size()} - 1 > \text{right.size()}$

⇒ shift largest from left to
right

1.

$\{2, 2, 2\}$

ms.erase(2)

$\hookrightarrow \{ \}$

2.

$\{2, 2, 2\}$

ms.extract(2)

ms.erase(ms.find(2))

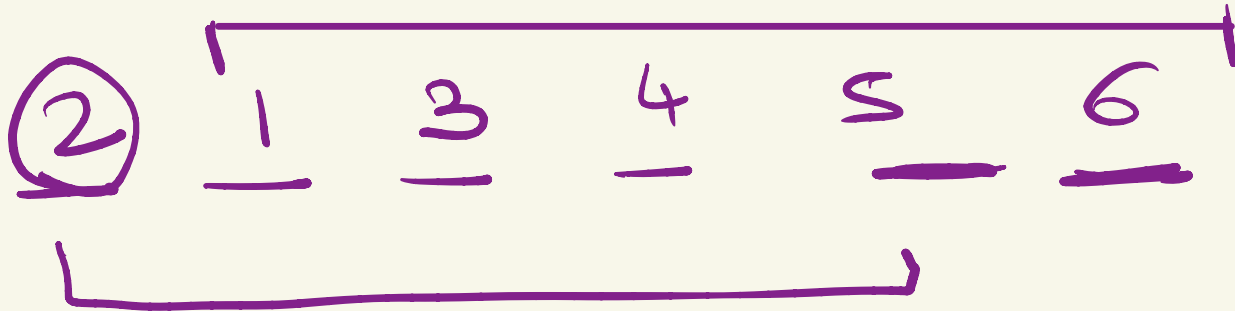
$\hookrightarrow \{2, 2\}$

$\{ 1, \boxed{2}, 3 \}$

$\{ 4, 5 \}$

.erase(2)

x priority-queue dont have .erase()



k=5

Approach 1 - Two Multisets

$\lceil \frac{k}{2} \rceil \rightarrow lo, hi$ $\rightarrow L^*, R^*$



TC $\rightarrow O(n \log k)$

```
vector<double> ans;
for (int i = 0; i < (int)nums.size(); i++) {
    // 1) Insert new element
    if (lo.empty() || nums[i] <= *lo.rbegin()) lo.insert(nums[i]);
    else hi.insert(nums[i]);
    balance();  $\leftarrow$  balance the 2 multisets

    // 2) Remove element that goes out of the window
    if (i >= k) {
        long long out = nums[i - k];
        if (out <= *lo.rbegin()) lo.erase(lo.find(out));
        else hi.erase(hi.find(out));
        balance();  $\leftarrow$  balance the 2 multisets
    }

    // 3) Compute median once we have k elements in the window
    if (i >= k - 1) {
        odd if (k % 2 == 1) ans.push_back(*lo.rbegin());
        even else ans.push_back(((double)*lo.rbegin() + (double)*hi.begin()) / 2.0);
    }
}
return ans;
```

insert
+ balance

remove
+ balance

$\downarrow$ iterator

Approach 1 - Two Multisets

good approach



multiset<long long> lo, hi;

auto balance = [&]() {

if (lo.size() > hi.size() + 1) {
hi.insert(*lo.rbegin());
lo.erase(prev(lo.end()));
}

if (hi.size() > lo.size()) {
lo.insert(*hi.begin());
hi.erase(hi.begin());
}

};

lambda

balance
lo $\rightarrow$ high

balance
hi $\rightarrow$ lo

$\underbrace{\quad \quad \quad}_\lambda \quad \rightarrow$

PBDS

→ bounded to competitive programming

↳ policy based data structures

↳ indexed ordered set

set

ms.find-by-order(index);

Approach 2 - Policy Based Data Structures



```
#include <ext/pb_ds/assoc_container.hpp>  
#include <ext/pb_ds/tree_policy.hpp>
```

```
using namespace __gnu_pbds;
```

```
template <class T>
```

```
using ordered_set = tree<  
    T,  
    null_type,  
    less<T>,  
    rb_tree_tag,  
    tree_order_statistics_node_update
```

```
>;
```

Template
code

ordered-set $\rightarrow$ set $\rightarrow$ multiset

set < pair > st

$\rightarrow$ {value, index}

2 2 2 $\rightarrow$ $\{ \underset{\uparrow}{2}, 0 \}, \{ \underset{\uparrow}{2}, 1 \}, \{ \underset{\uparrow}{2}, 2 \}$



```
class Solution {
public:
    vector<double> medianSlidingWindow(vector<int>& nums, int k) {
        ordered_set<pair<long long,int>> st; ← ordered set
        vector<double> ans;
        ans.reserve(nums.size() - k + 1);

        for (int i = 0; i < (int)nums.size(); i++) {
            // 1) Insert new element (nums[i], i)
            → st.insert({(long long)nums[i], i});

            // 2) Remove the element that slides out (i - k)
            if (i >= k) {
                st.erase({(long long)nums[i - k], i - k});
            }

            // 3) Once the window size is k, find the median
            if (i >= k - 1) {
                int size = k;
                int mid = size / 2;

                if (size % 2 == 1) {
                    // odd window: the median is the (k/2)-th element (0-based)
                    auto it = st.find_by_order(mid);
                    ans.push_back((double)it->first); ←
                } else {
                    // even window: average of the (k/2 - 1)-th and (k/2)-th elements
                    auto it1 = st.find_by_order(mid - 1);
                    auto it2 = st.find_by_order(mid);
                    ans.push_back(((double)it1->first + (double)it2->first) / 2.0);
                }
            }
        }
        return ans;
    }
};
```

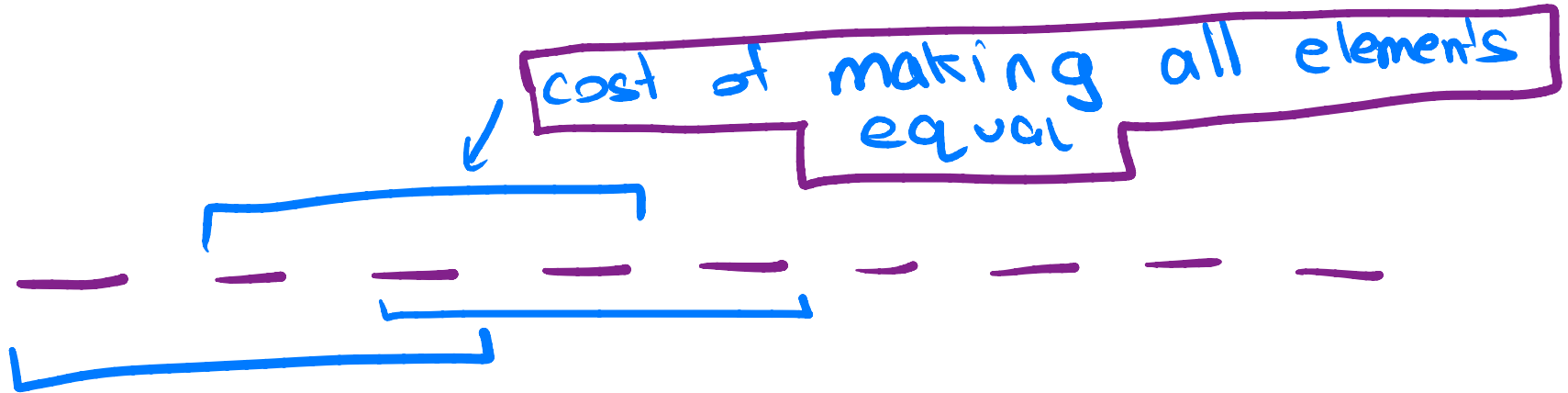

Problem 2 [CSES]

* Good problem



Sliding Window Cost:

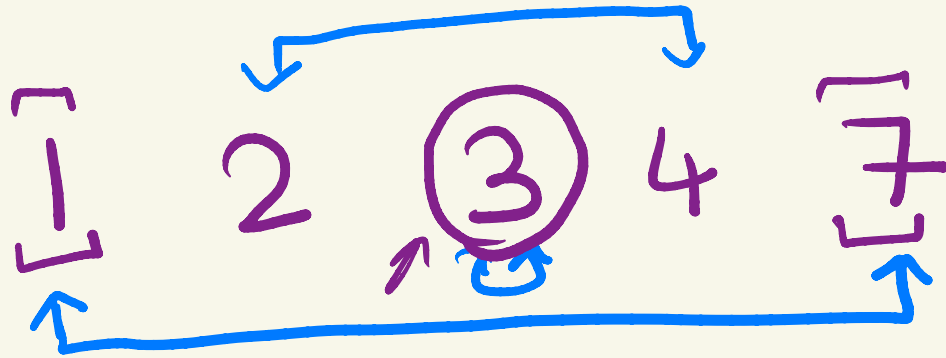
<https://cses.fi/problemset/task/1077>



equal, then you change them to the median

$\underline{7} \quad \underline{4} \quad \underline{1} \quad \underline{2} \quad \underline{3}$

Why??



[3]

2, 4 $\rightarrow$ 5

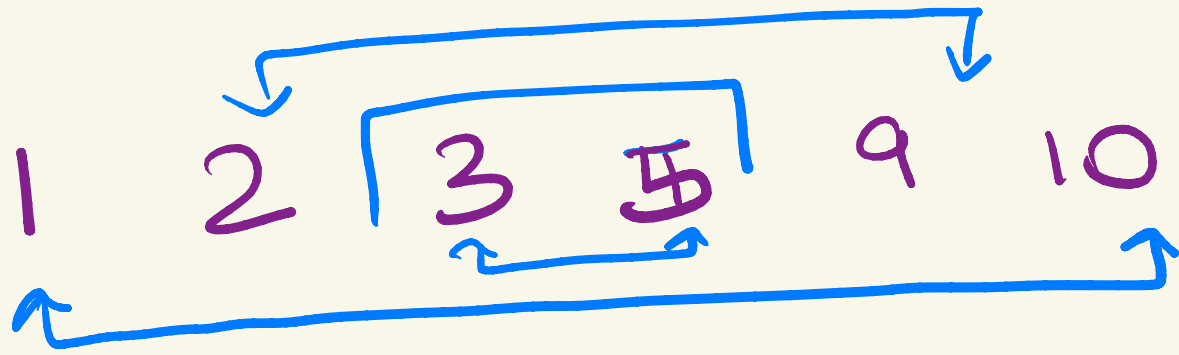
$(5-2) + (5-4) = 4$

2, 4 $\rightarrow$ 3 = 2

cost $1 \rightarrow x$ + cost $7 \rightarrow x$
 = constant

5 $\rightarrow$ 6 $(5-1) + (7-5)$

~~4~~ $\rightarrow$ 6 $(\underline{3}-1) + (\underline{7}-3)$



↳ [3, 5]
↑

→ median → 2 multisets

$$|4-2| + |3-4| + |4-4| \\ + |7-4| + |5-4|$$

$O(n)$

2 3 ④ 7 5

Sum = k * median ~~X~~

$$\rightarrow (3 * 4) - (2 + 3 + 4) + (5 + 7) - 2 * 4$$

2, 3, 4

Left

$$+ 4 - 2$$

$$4 - 3$$

$$4 - 4$$

$$3 * 4 - (2 + 3 + 4)$$

5, 7

high

$$5 - 4$$

$$\rightarrow 7 - 4$$

$$(5 + 7) - 2 * (4)$$

ms Left, Right,
int LeftSum, RightSum

$$\begin{aligned} & \text{Left.size() * median} - \text{Leftsum} \\ & + \text{rightSum} - \text{rightsize() * median} \end{aligned}$$

→ maintaining median

→ left & right multisets sum

??


```
signed main() {  
    int n, k; cin >> n >> k;  
  
    vector<int>a(n);  
    for(auto &i : a) cin >> i;  
  
    MedianContainer container;  
  
    for(int i = 0; i < k; i++) container.insert(a[i]);  
  
    cout << container.cost() << " ";  
  
    for(int i = k; i < n; i++) {  
        container.remove(a[i - k]);  
        container.insert(a[i]);  
        cout << container.cost() << " ";  
    }  
  
    return 0;  
}
```

[first try
yourself!]

template



<https://cses.fi/paste/f7e67e8aa631b995a98236/>

Problem 3 [CSES]

→ Thinking



[Maximum Subarray Sum 2:]

<https://cses.fi/problemset/task/1644>

Man. Croz!

Hint → [Prefix sum]



L - R

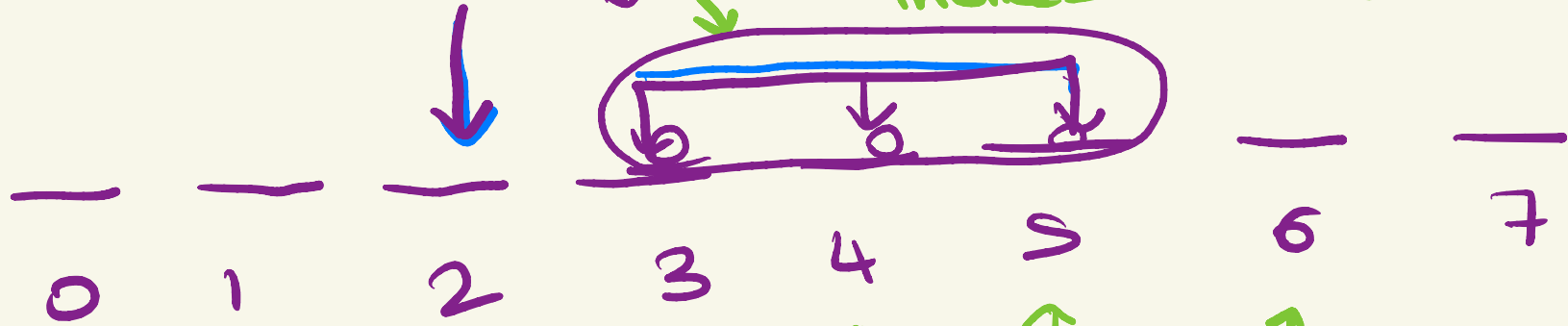
$pre[r] - pre[l-1]$

→ we want max subarray sum
[a, b]

max element! [a = 3, b = 5]

t

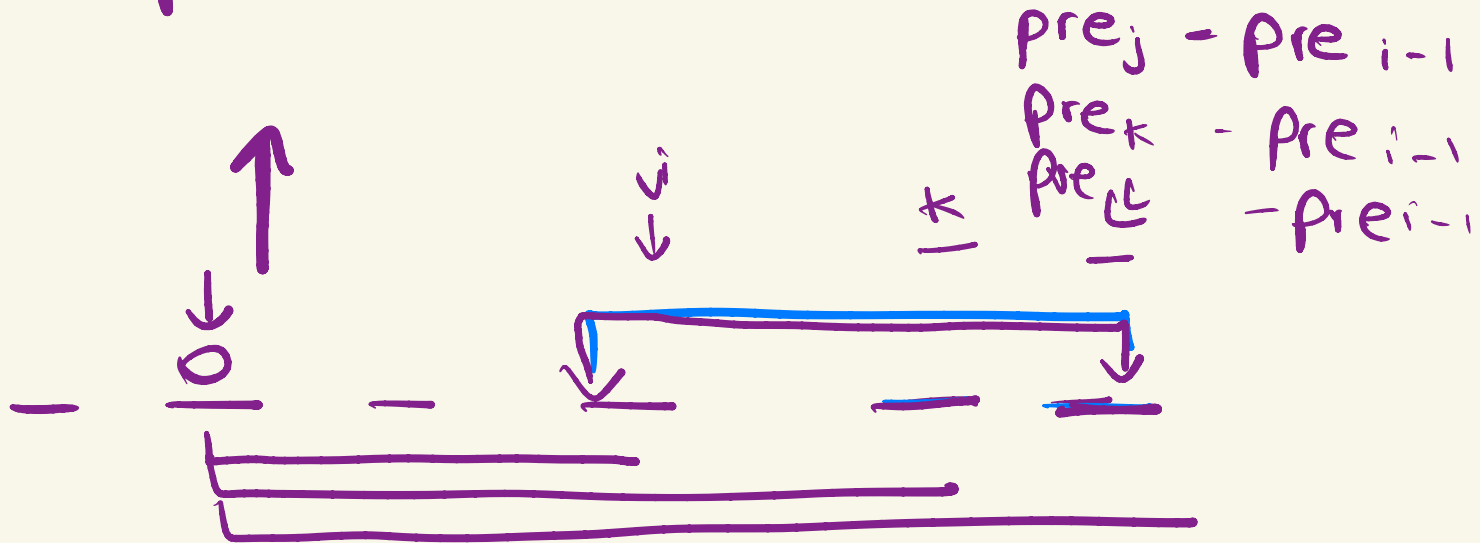
↓ window of right
indices is fixed



↑ $\boxed{\text{pref}[R]} - \text{pref}[i-1]$

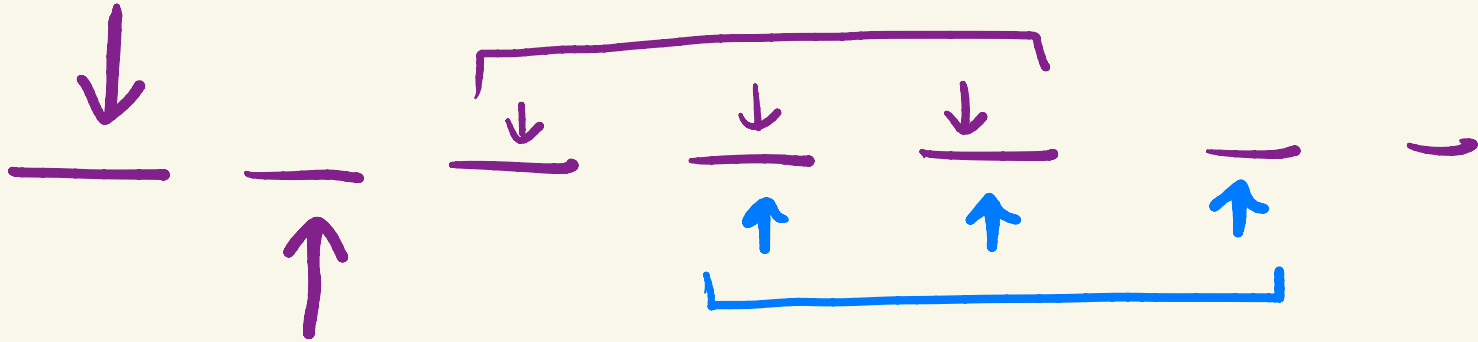
← max

* if we fix the left index
we have a window for
possible indices



$[a, b]$

$[3, 5]$



Implement this problem

Task